DOCUMENTATION · COMPUTER VISION ASSIGNMENT

# Scoreboard Data Extraction From Video

Screenshots of the input, the pipeline running, the scoreboard being detected, and the extracted output, with a short explanation for each.

| Input | Ground-truth accuracy | Full-video result |
| --- | --- | --- |
| bowling_scoreboard.mp4 · 58s · 1920×1080 | 34 / 34 cells (frame 1) | 5 / 6 states clean |

## 01 Input frame

A representative frame sampled from the source video (extracted at 3fps via ffmpeg). The scoreboard graphic is a fixed on-screen overlay: a lane number, the active player's name, a 10-frame grid per player, and a team-total row along the bottom.

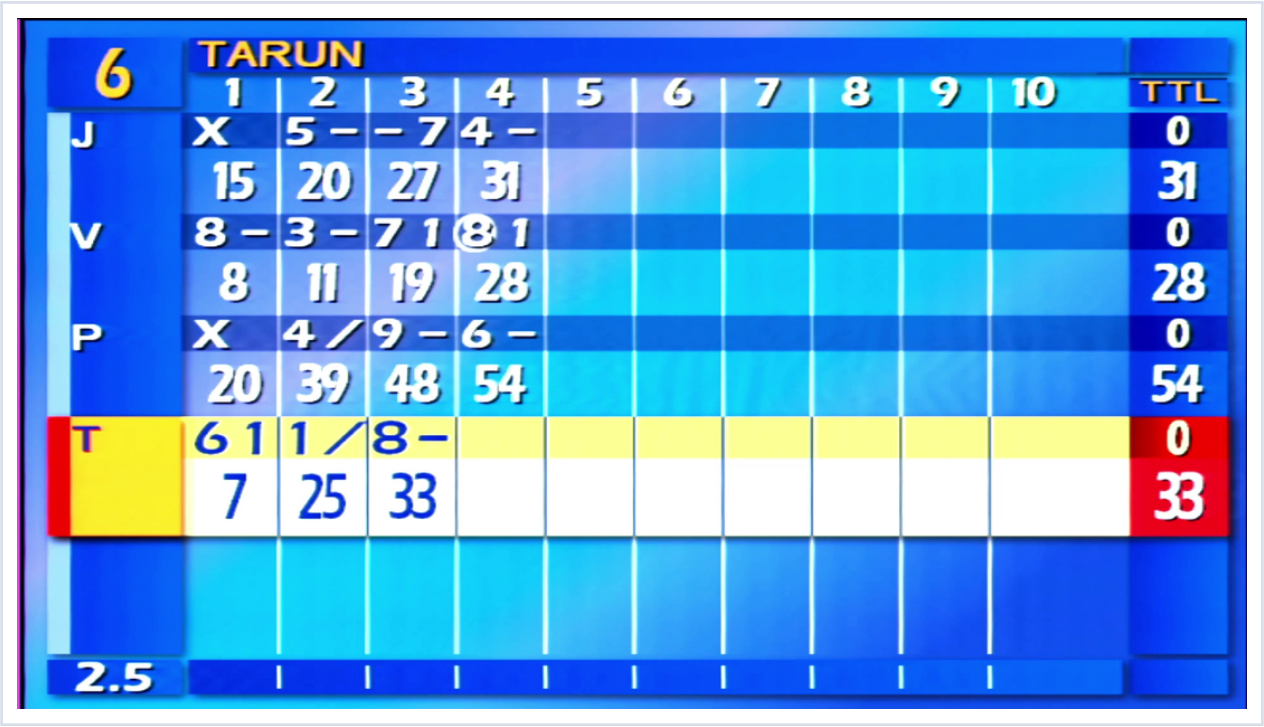


Fig. 1 — frame_000001.png, the first stable scoreboard frame in the video.

## 02 Code running

The full pipeline runs as a single command once frames are extracted. It samples the video, discards non-scoreboard frames (the broadcast cuts to bumpers/cartoons between scoreboard segments), recognizes every cell, cross-checks the result against bowling scoring rules, and writes structured JSON.

```
Command Prompt

$ cd src
$ python pipeline.py --frames ../data/frames --out ../output/scorecards.json

174 frames -> 26 stable segments -> 26 picked
Wrote 6 scorecard snapshots to ../output/scorecards.json (20 non-scoreboard frames skipped)

$
```

Fig. 2 — actual output of `python pipeline.py`: 174 extracted frames collapse to 26 stable segments, of which 6 are distinct scorecard states (the rest are non-scoreboard footage, correctly skipped).

---

## 03 Scoreboard detected/extracted

Every cell boundary is calibrated against the graphic's real pixel geometry (measured via gridline detection, not eyeballed). The green overlay below shows exactly which region is cropped and fed to the recognizer for every one of the 88 cells in the grid.

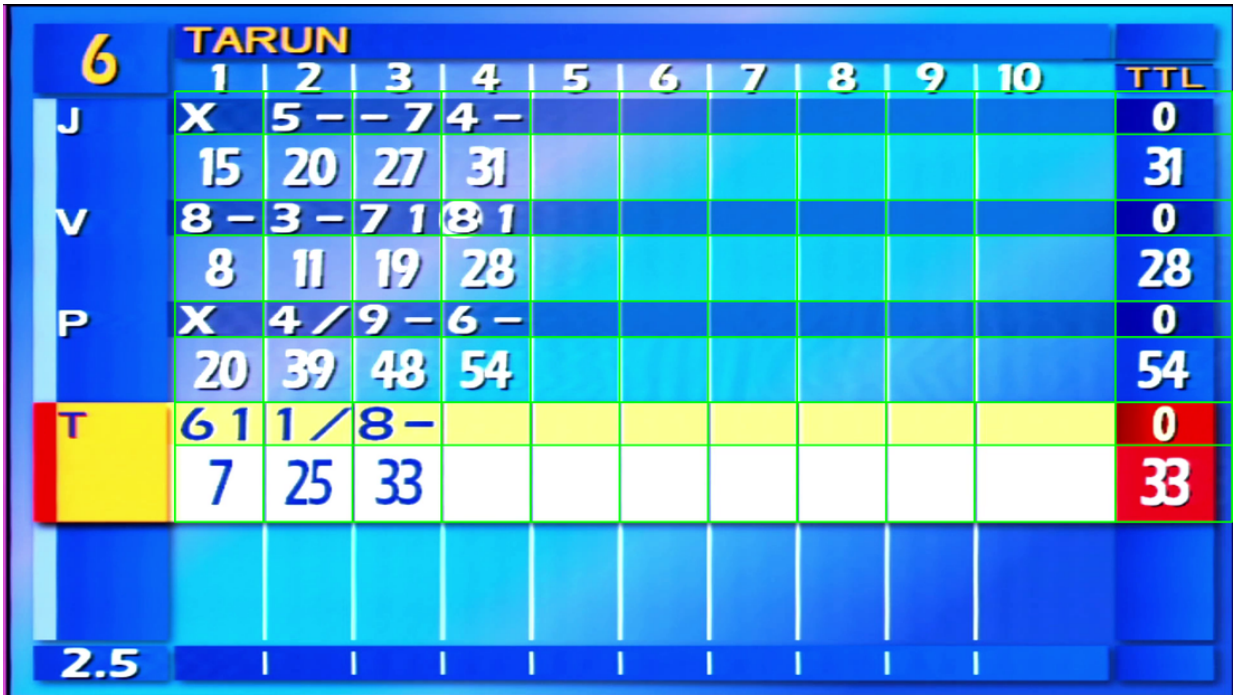


Fig. 3 — the calibrated cell grid overlaid on the input frame.

The system also detects a transient animation the broadcast pops up after most rolls, which otherwise corrupts whatever cells it covers. Detection is per row-block against a known clean-background baseline (not a fixed box), since the animation isn't pinned to one screen position.

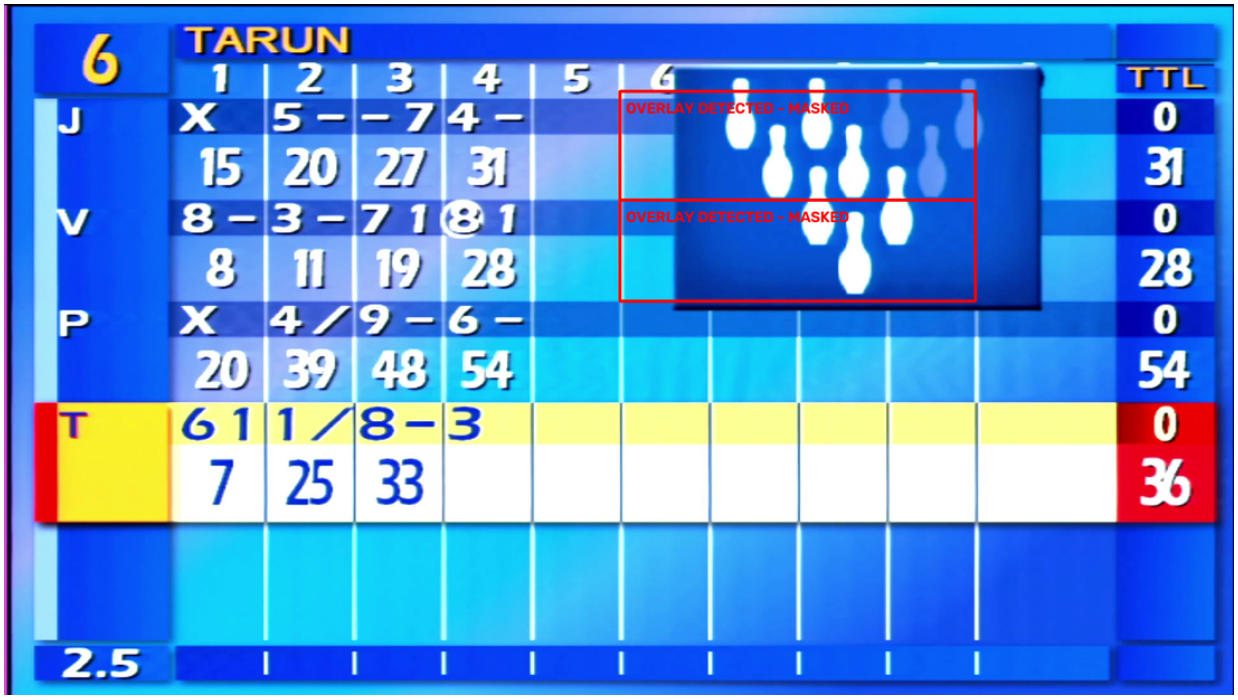


Fig. 4 — the overlay correctly detected and masked for the J and V rows in this frame; those cells are left blank rather than mis-recognized.

---

## 04 Extracted output

The final output is one structured JSON entry per distinct scorecard state found in the video — every player's per-frame rolls, running totals, lane number, and active player name. A `validation_issues` block flags anything a cross-check against bowling scoring rules found suspect.

output/scorecards.json

```
{
  "frame_file": "frame_000001.png",
  "state": "none",
  "lane": "6",
  "name": "TARUN",
  "players": {
    "J": {
      "symbols": ["X", "5-", "-7", "4-", "", "", "", "", "", ""],
      "scores":  [15, 20, 27, 31, null, null, null, null, null, null],
      "ttl": 31
    },
    "V": { ... }, "P": { ... }, "T": { ... }
  },
  "validation_issues": {}
}
```

Fig. 5 — excerpt of output/scorecards.json for the frame shown in Fig. 1 (player J shown; V, P, T follow the same structure). Every value shown matches the source frame exactly.

Full development history — every problem hit, what changed, and why — is in `docs/BUILD_LOG.md` in the repository.